Dynamical Tracking

Louis Urbin
Metrics of curves in shape optimization and analysis
Scuola Normale Superiore

July 25, 2025

Notations and Objectives

Goal: track a geometric shape in a video. This is equivalent to dynamically following a closed curve using Sobolev active contour segmentation.

Relevant spaces of curves:

- $\mathcal{M}_i$: space of immersed curves,
- $\mathcal{M}_d$: curves normalized to zero center and unit length (invariant under translation and scaling),
- $\mathrm{St}(L^2, 2)$: Stiefel manifold, with an isometric surjection onto $\mathcal{M}_d$.

Tracked dynamic quantities:

- $\mu_k \in \mathrm{St}(L^2, 2)$: curve representation in Stiefel (frame),
- $\nu_k \in T_{\mu_k}\mathrm{St}(L^2, 2)$: corresponding velocity,
- $\xi_k \in \mathbb{R}^3$: center and length (cenlen structure),
- $\chi_k \in \mathbb{R}^3$: cenlen velocity.

Three main algorithmic steps: prediction, estimation, correction.
Note: noise can be added at each stage, as in the Python implementation.

Step 1: Prediction

From μ_{k-1}, ν_{k-1}, we predict:

$$\hat{\mu}_k = \text{geodesic shooting from } \mu_{k-1} \text{ with velocity } \nu_{k-1}$$

$$\hat{\nu}_k = \text{parallel transport of } \nu_{k-1} \text{ along the geodesic}$$

For the cenlen structure, from ξ_{k-1}, χ_{k-1}, we predict:

$$\hat{\xi}_k = \xi_{k-1} + \chi_{k-1}\Delta t, \qquad \hat{\chi}_k = \chi_{k-1}$$

The update of χ_k will be done during the correction step. In our case, we use $\Delta t = 1$.

Implementation — Prediction

```python
# Check frame dimensions
assert mu_history[k-1].nc == nu_history[k-1].nc, "Frame dimensions are not compatible."
assert mu_history[k-1].nr == nu_history[k-1].nr, "Frame dimensions are not compatible."

mu_pred, _ = SC.shoot_geodesic_w_vel(mu_history[k-1], nu_history[k-1], 0.3)  # geodesic evolution

# Compute the parallel transport of nu_{k-1} along the geodesic from mu_{k-1} to mu_k
_, _, nu_pred = SC.Stiefel_minimal_geodesic(mu_history[k-1], mu_pred, 1., precision=1e-3)

# Verify that mu_pred is on the Stiefel manifold and that nu_pred is in the tangent space at mu_pred
mu_pred = SC.frame_project(mu_pred)
SC.frame_project_on_tan_Stiefel(nu_pred, mu_pred)

#print("nu_pred norm :", SC.frame_norm(nu_pred))  # Check the norm of the predicted velocity

# Compute the predicted cenlen values
xi_pred = SC.cenlen()
xi_pred.cen.x = xi_history[k-1].cen.x + chi_history[k-1][0]
xi_pred.cen.y = xi_history[k-1].cen.y + chi_history[k-1][1]
xi_pred.len = xi_history[k-1].len + chi_history[k-1][2]

# Update eta_k, omega_k (noise for mu and xi)
eta_k = SC.random_direction_on_tangent(mu_pred, sigma=0.01, alpha=1.5, firstfreq=0, lastfreq=-1, K=-1)
omega_k = 1e-2 * np.random.randn(3)  # noise in R3

# Update velocity with noise
nu_corr = SC.matrix_sum(nu_pred, eta_k)
chi_corr = chi_history[k-1] + omega_k
```

Step 2: Estimation

- From $\hat{\mu}_k$ and $\hat{\xi}_k$, we reconstruct a curve $\hat{c}_k \in \mathcal{M}_i$ using `cenlen_transform`.
- Given this predicted curve and the image at frame k, the Sobolev algorithm returns an estimate $\tilde{c}_k$ of the actual contour.
- We then extract:

$$\tilde{\mu}_k, \quad \tilde{\xi}_k$$

Implementation — Estimation

```
# Observation via active segmentation (Sobolev active contour)
curve_k_estim = SC.frame_to_Md_curve(mu_pred)

SC.cenlen_transform(curve_k_estim, xi_pred)  # Convert mu_pred, xi_k to a curve in Mi

y_k, cl_k = TF.sobolev_active_contour(video[k], curve_k_estim, scale=0.01, steps=100, unit=100, mask_by_winding_p=False)
```

Step 3: Correction

- Correction in Stiefel:

$$\nu_k = \hat{\nu}_k + K_\nu \cdot \log_{\hat{\mu}_k}(\tilde{\mu}_k)$$

- Cenlen correction:

$$\chi_k = \hat{\chi}_k + K_\chi(\tilde{\xi}_k - \hat{\xi}_k), \quad \xi_k = \hat{\xi}_k + K_\xi(\tilde{\xi}_k - \hat{\xi}_k)$$

Implementation — Correction

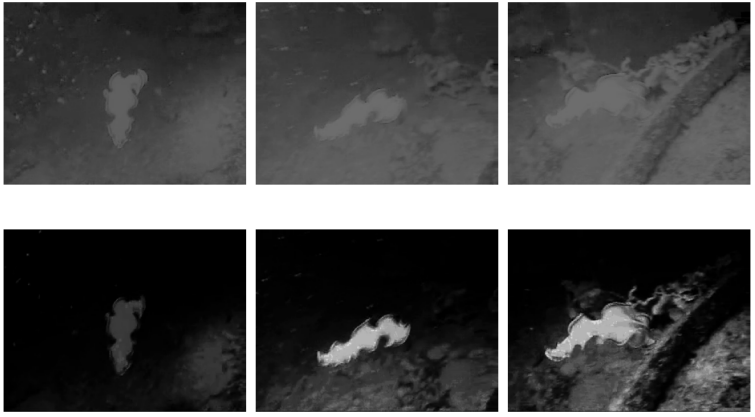
```
# Filter correction
log_nu = TF.logarithmic_map(mu_pred, y_k)

log_chi = np.zeros(3)
log_chi[0] = cl_k.cen.x - xi_pred.cen.x
log_chi[1] = cl_k.cen.y - xi_pred.cen.y
log_chi[2] = cl_k.len - xi_pred.len

# Filter gains
K_nu = 0.8  # mu filter gain
SC.matrix_scalar(log_nu, K_nu)

K_chi = 0.3  # chi filter gain
log_chi *= K_chi

K_xi = 0.6  # xi filter gain
xi_pred.cen.x = xi_pred.cen.x + K_xi * (cl_k.cen.x - xi_pred.cen.x)
xi_pred.cen.y = xi_pred.cen.y + K_xi * (cl_k.cen.y - xi_pred.cen.y)
xi_pred.len   = xi_pred.len   + K_xi * (cl_k.len   - xi_pred.len)

nu_filtered = SC.matrix_sum(nu_corr, log_nu)  # Filtered velocity
SC.frame_project_on_tan_Stiefel(mu_pred, nu_filtered)  # Project the filtered velocity onto the tangent space of the Stiefel manifold
chi_filtered = chi_corr + log_chi  # Filtered cenlen values

mu_pred = SC.frame_project(mu_pred)  # Project mu_pred onto the Stiefel manifold
SC.frame_project_on_tan_Stiefel(nu_filtered, mu_pred)  # Project the filtered velocity onto the tangent space of the Stiefel manifold

# Save state
mu_history.append(mu_pred)
nu_history.append(nu_filtered)
xi_history.append(xi_pred)
chi_history.append(chi_filtered)
```

Why correct ξ_k?

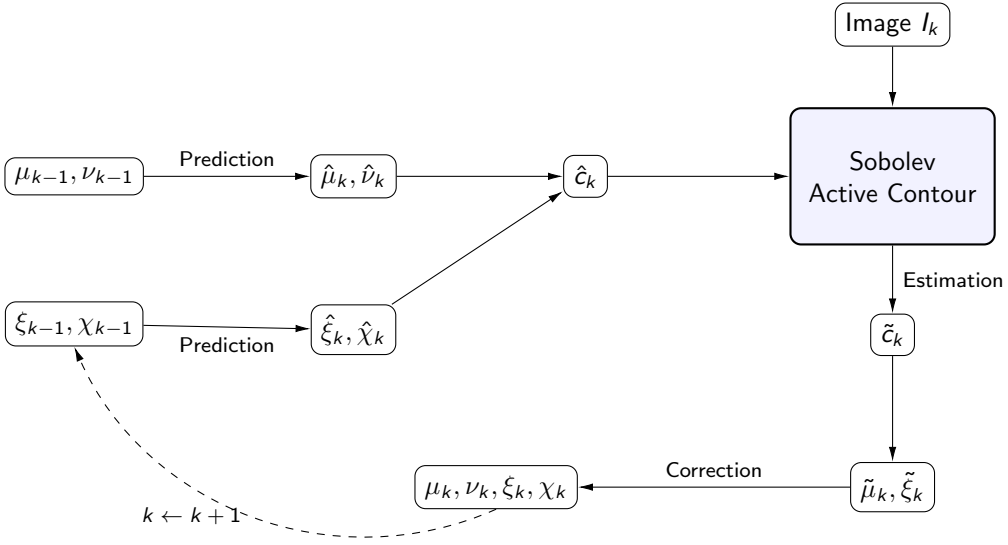
- Easy to implement,
- Significantly improves the stability of the cenlen tracking.
- See the two videos `one_gain.avi` and `two_gains.avi` to observe the difference. The red curve shows the prediction, while the green curve represents the observation obtained via the Sobolev active contour.

Video Preprocessing

Before applying the tracking algorithm, a preprocessing step is used to enhance curve visibility (mainly by increasing contrast).



Algorithm Overview



Results

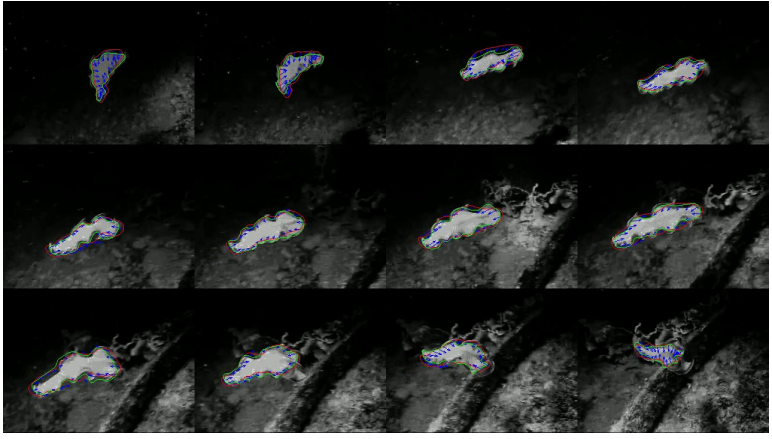


Figure: Shape tracking results. Red: predicted contour (μ, ξ); Green: observed contour using Sobolev active contour (y, cl); Blue: vector field (ν, χ).